NLP Phishing Email Detection

Detecting Malicious Emails Using NLP

Course: Selected Topics in Data Science

Project Type: NLP — Email Filter

Submitted to: Dr.Alaa Mahmoud Hamdy

Student Name: Mohamed Khallaf

Student ID: 22P0245

Student Name: Roaa Gadara

Student ID: 22p0188

Submission Date: May 2026

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Dataset **3**
3.1 Source and Description . . . 3
3.2 Class Distribution . . . 3

4 Methodology **4**
4.1 Text Preprocessing . . . 4
4.2 Feature Engineering . . . 4
4.3 Tokenization and Sequence Padding . . . 5

5 Models **6**
5.1 Baseline Model 1 — Naive Bayes . . . 6
5.2 Baseline Model 2 — Logistic Regression . . . 6
5.3 Main Model — Bidirectional LSTM . . . 6

6 Results **7**
6.1 Training History . . . 7
6.2 Confusion Matrix . . . 8
6.3 Classification Report . . . 8
6.4 Model Comparison . . . 9

7 Explainability **10**
7.1 Global Feature Importance . . . 10
7.2 Per-Email Explanation . . . 10

8 Discussion **11**
8.1 Why Bidirectional LSTM Works Well . . . 11
8.2 Limitations . . . 11
8.3 Future Improvements . . . 11

9 Conclusion **12**

1. Introduction

Phishing is one of the most common and dangerous types of cyber attacks today. The idea behind it is simple but effective — an attacker sends an email that looks like it is coming from a trusted source, like a bank or a company, and tricks the user into clicking a link or giving away their password or personal information.

What makes phishing different from regular spam is that it is not just annoying advertising — it is a targeted attack designed to steal something from the user. A spam filter that blocks emails containing certain words is not enough, because phishing emails are carefully written to look legitimate and avoid simple filters.

This is where Natural Language Processing (NLP) becomes useful. Instead of just looking for specific words, NLP models can understand the meaning and structure of text, which makes them much better at catching emails that are trying to manipulate the reader.

In this project, we built a complete phishing email detection system using a Bidirectional Long Short-Term Memory (BiLSTM) neural network. We also trained two simpler baseline models — Logistic Regression and Naive Bayes — to compare their performance and show that deep learning provides a real advantage for this task.

The system was trained and tested on a real-world dataset of approximately 18,600 emails collected from Kaggle. The final model achieved an accuracy of **97.16%** on the test set.

2. Problem Statement

The goal of this project is to build a binary classification system that takes an email as input and outputs one of two labels:

- **Phishing Email** — the email is malicious and trying to deceive the reader
- **Safe Email** — the email is legitimate

The system needs to detect common phishing patterns such as:

- Impersonation of banks, companies, or organizations
- Urgency language such as "your account will be suspended"
- Requests for passwords, personal details, or financial information
- Suspicious links embedded in the email body

The main challenge is that phishing emails are designed to look legitimate. A simple keyword-based filter would not work well because phishing emails often avoid obvious red-flag words on purpose.

3. Dataset

3.1. Source and Description

The dataset used in this project is the *Phishing Email Detection* dataset published on Kaggle by subhajournal. It can be accessed at:

https://www.kaggle.com/datasets/subhajournal/phishingemails

The dataset contains **18,650 emails** with two columns:

- `Email Text` — the raw text content of the email
- `Email Type` — the label, either *Phishing Email* or *Safe Email*

3.2. Class Distribution

The dataset is slightly imbalanced, with more safe emails than phishing emails. Figure 1 shows the distribution.

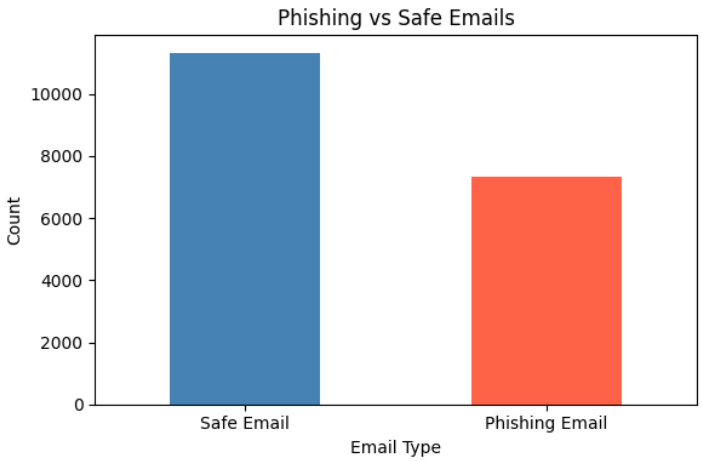

Figure 1: Distribution of Phishing vs Safe emails in the dataset

After cleaning (removing rows with missing values and invalid labels), the final dataset contained:

- **Safe Emails:** approximately 11,300
- **Phishing Emails:** approximately 7,300

The dataset was split into **80% training** and **20% testing**, using stratified sampling to ensure both splits had the same class ratio (approximately 60% safe, 40% phishing).

4. Methodology

4.1. Text Preprocessing

Before feeding the emails into any model, we applied a series of cleaning steps to normalize the text. Raw emails often contain HTML tags, URLs, symbols, and inconsistent formatting that would confuse the model if left in.

The cleaning pipeline applied to each email was:

1. Convert all text to **lowercase**
2. Replace all URLs (starting with `http` or `www`) with the token `url`
3. Remove all HTML tags using a regular expression
4. Remove all non-alphabetical characters (numbers, punctuation, symbols)
5. Collapse multiple whitespace characters into single spaces

Replacing URLs with a token rather than removing them completely was a deliberate choice — the presence of a URL in an email is itself a useful signal, but the specific URL content is not.

4.2. Feature Engineering

In addition to the raw text, we extracted six numerical features from each email before cleaning. These features capture characteristics of the email that are known to differ between phishing and safe emails.

Table 1: Engineered features and their descriptions

Feature	Description
`url_count`	Number of URLs found in the email
`email_length`	Total number of characters in the email
`urgency_count`	Count of urgency-related keywords (e.g. "click here", "verify", "suspended", "password")
`html_tags`	Number of HTML tags present in the email
`exclamations`	Number of exclamation marks
`dollar_signs`	Number of dollar sign characters

Figure 2 shows how these features differ between phishing and safe emails on average.

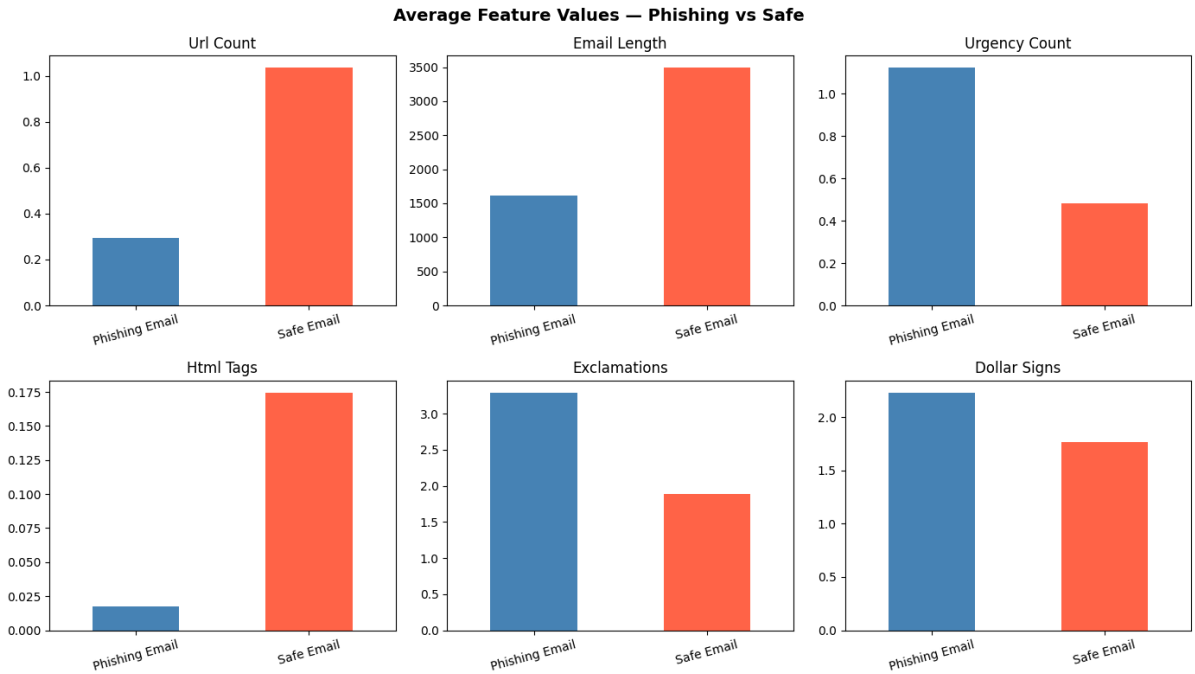

Figure 2: Average feature values for Phishing vs Safe emails

Some interesting observations from the features:

- Phishing emails had more **urgency keywords** (1.12 vs 0.48 on average) — consistent with manipulation tactics
- Safe emails were significantly **longer** (3,493 vs 1,614 characters) — real business emails tend to be more detailed
- Phishing emails had more **exclamation marks** (3.30 vs 1.89) — a common pattern in deceptive writing
- Safe emails had more **URLs** (1.04 vs 0.29) — likely because legitimate newsletters and business emails contain many footer links

4.3. Tokenization and Sequence Padding

For the deep learning model, the cleaned text was converted into numerical sequences using Keras' `Tokenizer`. The tokenizer was fitted only on the training data to avoid any data leakage into the test set.

- **Vocabulary size:** 50,000 most frequent words
- **Sequence length:** 300 tokens (padded or truncated)
- **Out-of-vocabulary token:** `<OOV>`

5. Models

We trained three models of increasing complexity to compare their performance and justify the use of deep learning.

5.1. Baseline Model 1 — Naive Bayes

Naive Bayes is one of the simplest and fastest text classification algorithms. It works by calculating the probability that each word belongs to each class and multiplying these probabilities together. Despite its simplicity, it often performs surprisingly well on text data.

We used a **Multinomial Naive Bayes** classifier with a TF-IDF vectorizer (up to 20,000 features, unigrams and bigrams). This model serves as the weakest baseline.

5.2. Baseline Model 2 — Logistic Regression

Logistic Regression is a stronger but still traditional machine learning model. It learns a weight for each word in the vocabulary and predicts the class based on the weighted sum of word frequencies.

We used Logistic Regression with a TF-IDF vectorizer (20,000 features, sublinear TF scaling, unigrams and bigrams). This model is interpretable — we can directly read off which words push the model toward predicting phishing.

5.3. Main Model — Bidirectional LSTM

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) specifically designed to learn from sequences. Unlike traditional models that treat text as a bag of words, LSTM reads the email word by word and remembers context across the entire sequence.

A **Bidirectional** LSTM reads the text in both directions — left to right and right to left — which allows it to capture context from both sides of each word. This is particularly useful for detecting phishing, where the meaning of a phrase can depend on what comes before and after it.

The architecture used in this project is:

Table 2: Bidirectional LSTM model architecture

Layer	Type	Output Shape	Parameters
1	Embedding	(None, 300, 128)	6,400,000
2	Bidirectional LSTM	(None, 128)	98,816
3	Dropout (0.3)	(None, 128)	0
4	Dense (ReLU)	(None, 64)	8,256
5	Dense (Sigmoid)	(None, 1)	65

The model was trained using:

- **Optimizer:** Adam
- **Loss function:** Binary crossentropy
- **Batch size:** 128
- **Maximum epochs:** 10
- **Early stopping:** Patience of 2 epochs, monitoring validation loss
- **Validation split:** 10% of training data

Early stopping was used to prevent overfitting. It automatically stops training when the validation loss stops improving and restores the best weights.

6. Results

6.1. Training History

Figure 3 shows the accuracy and loss curves during LSTM training. The model converged quickly — reaching above 97% validation accuracy by the second epoch. Early stopping triggered after epoch 4, restoring the best weights from epoch 2.

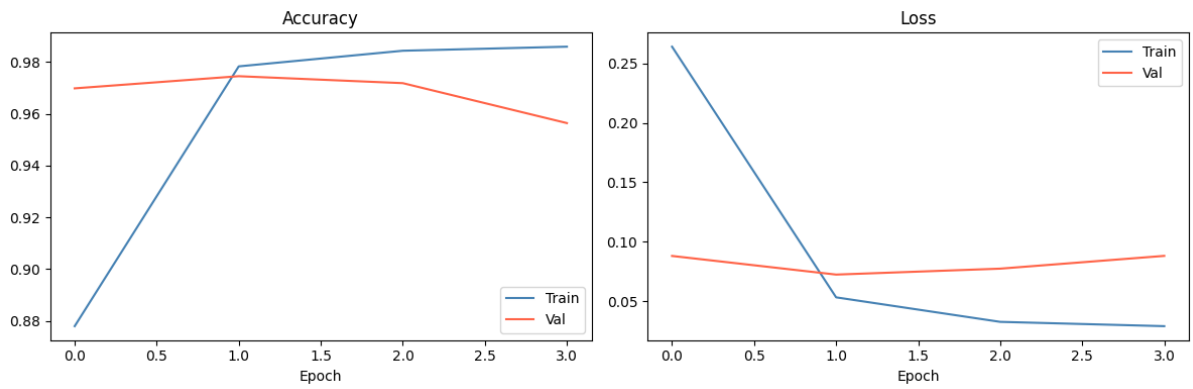

Figure 3: LSTM training accuracy and loss curves

6.2. Confusion Matrix

Figure 4 shows the confusion matrix for the LSTM model on the test set of 3,727 emails.

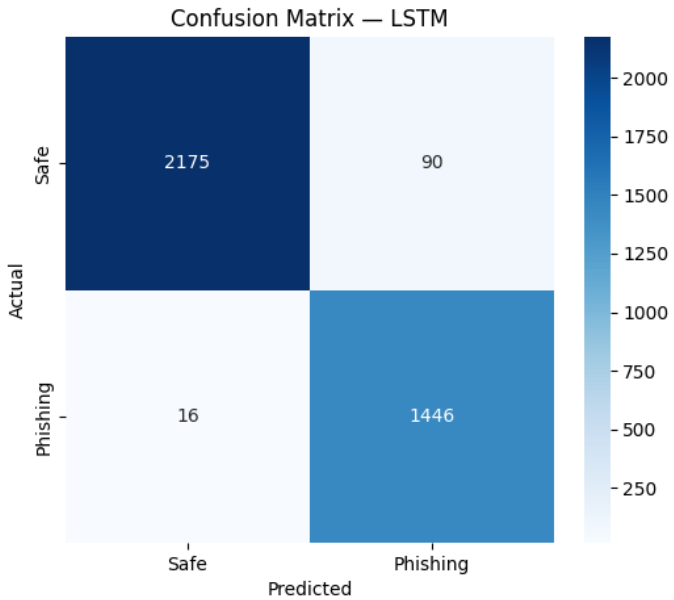

Figure 4: Confusion matrix for the Bidirectional LSTM model

- **2,175** safe emails correctly classified (True Negatives)
- **1,446** phishing emails correctly detected (True Positives)
- **90** safe emails incorrectly flagged as phishing (False Positives)
- **16** phishing emails missed (False Negatives)

The most important number here is the **False Negative count of only 16** — meaning the system missed only 16 actual phishing emails out of 1,462. This gives a phishing recall of 98.9%, which is excellent for a security application.

6.3. Classification Report

Table 3 shows the full classification report for the LSTM model.

Table 3: Classification report — Bidirectional LSTM

Class	Precision	Recall	F1-Score	Support
Safe Email	0.99	0.96	0.98	2,265
Phishing Email	0.94	0.99	0.96	1,462
Accuracy			**0.97**	**3,727**
Macro Avg	0.97	0.97	0.97	3,727
Weighted Avg	0.97	0.97	0.97	3,727

6.4. Model Comparison

Figure 5 and Table 4 compare the accuracy of all three models.

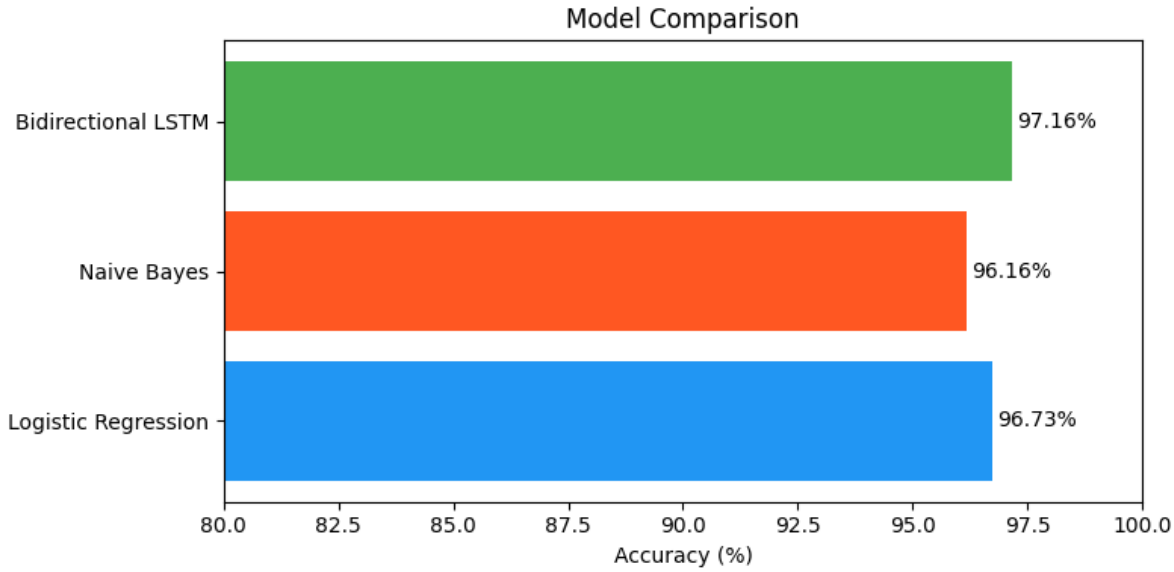

Figure 5: Accuracy comparison across all three models

Table 4: Model accuracy comparison

Model	**Type**	**Accuracy**
Naive Bayes	Traditional ML	96.16%
Logistic Regression	Traditional ML	96.73%
Bidirectional LSTM	**Deep Learning**	**97.16%**

The results show a clear progression: each more sophisticated model outperforms the previous one. While the improvement from Logistic Regression to LSTM may seem small (0.43%), in a security context this difference translates to catching significantly more phishing emails that the simpler model would have missed.

7. Explainability

One limitation of deep learning models is that they are difficult to interpret — you cannot easily explain why the model made a specific decision. To address this, we used the Logistic Regression model to provide explainability, since its weights directly correspond to how much each word influences the prediction.

7.1. Global Feature Importance

Figure 6 shows the top 15 words that push the model toward predicting *Phishing* and the top 15 words that push toward *Safe*.

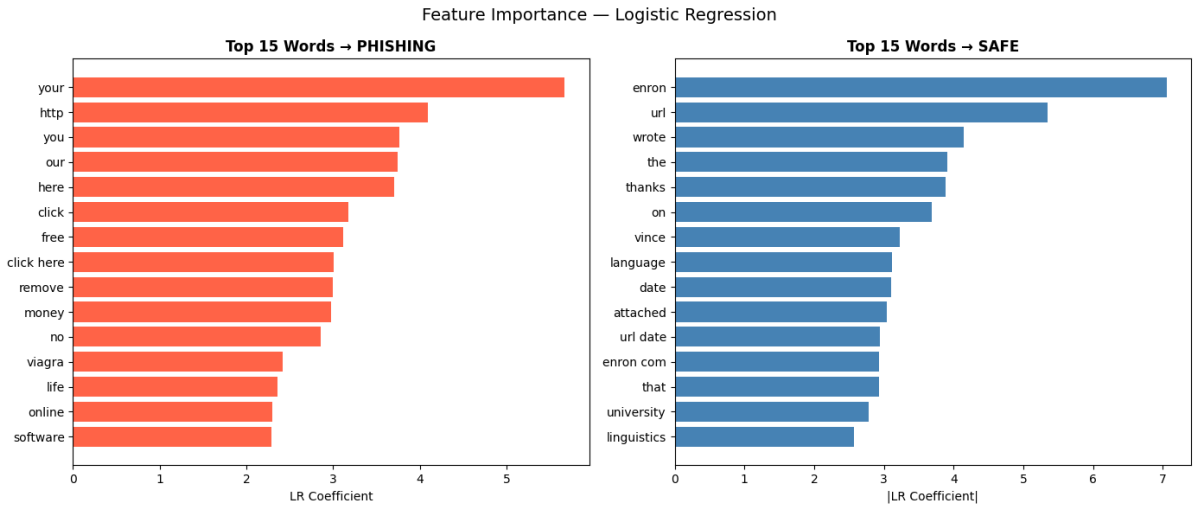

Figure 6: Top 15 words driving Phishing vs Safe predictions

The phishing words — such as *your*, *click here*, *free*, *money*, and *verify* — align perfectly with known phishing tactics. The safe words — such as *wrote*, *thanks*, *university*, and *linguistics* — reflect the vocabulary of normal business and academic emails.

7.2. Per-Email Explanation

For each individual email, the system can also explain which specific words in that email contributed most to the prediction. For example, for the custom phishing email:

"Dear user, your account has been suspended. Click here immediately to verify your details and avoid permanent suspension."

The top contributing words were: `your` (+0.97), `click here` (+0.47), `click` (+0.44), `here` (+0.39), `account` (+0.25), `your account` (+0.25) — all strongly pushing toward phishing. The model predicted this email as phishing with **99.94% confidence**.

8. Discussion

8.1. Why Bidirectional LSTM Works Well

The key advantage of the Bidirectional LSTM over simpler models is its ability to understand **context**. A word like "account" is not inherently phishing-related, but the phrase "your account has been suspended" in the context of a request to click a link is a strong phishing signal. The LSTM can capture this kind of multi-word pattern across the full email.

8.2. Limitations

Despite the strong results, there are some limitations worth noting:

- **Dataset bias:** The dataset contains a large proportion of adult content phishing emails. In a real deployment, the model should be tested on a more diverse range of phishing types.
- **No header analysis:** The system only analyzes the email body, not the sender address, subject line, or routing headers — all of which carry useful signals.
- **Static model:** Phishing tactics evolve over time. A deployed system would need to be retrained periodically on new data.
- **Language:** The model was trained on English emails only.

8.3. Future Improvements

- Using a **pre-trained transformer model** such as BERT, which has much deeper language understanding
- Adding **URL analysis** — checking whether embedded links point to suspicious domains
- Combining **email header features** with text features
- Building a **real-time web interface** where users can paste an email and get an instant prediction

9. Conclusion

In this project, we built a phishing email detection system using NLP and deep learning. We trained three models on a real-world dataset of approximately 18,600 emails and compared their performance.

The main model — a Bidirectional LSTM — achieved **97.16% accuracy** on the test set, correctly identifying 98.9% of all phishing emails. This outperformed both baseline models (Naive Bayes at 96.16% and Logistic Regression at 96.73%), confirming that the added complexity of deep learning is justified for this task.

We also demonstrated explainability by identifying the most influential words for each prediction, making the system's decisions transparent and interpretable.

The results show that NLP-based phishing detection is a viable and effective approach, significantly more sophisticated than traditional keyword-based spam filters.

References

1. subhajournal. *Phishing Email Detection Dataset*. Kaggle, 2022. https://www.kaggle.com/datasets/subhajournal/phishingemails
2. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735–1780.
3. Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681.
4. Chollet, F. et al. (2015). *Keras*. https://keras.io
5. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.